FUNDAMENTALS IN AI and ML

COURSE CODE: CSA2001

Name : Tushita Pathak

Registration No. : 25BAI10339

Chess Engine

1. Project Abstract

I built this Chess Engine as a personal deep-dive into classical AI — the kind of search-and-evaluation approach that dominated competitive chess programming for decades before neural networks entered the picture. The end result is a fully playable web application where a human player competes against an AI opponent that I wrote from scratch in Python.

The backend runs on Flask and uses the python-chess library to handle legal move generation and board state. The AI itself is driven by the Minimax algorithm with Alpha-Beta Pruning, supplemented by Quiescence Search to prevent the engine from making tactically naive evaluations at the edge of its search horizon. The evaluation function combines standard material counting with Piece-Square Tables — lookup tables that encode positional preferences for each piece type — and a separate endgame heuristic that changes how the King is valued once the heavy pieces come off the board.

The frontend is delivered as a single self-contained HTML page with embedded JavaScript, so there is nothing to install on the client side. A user types a move in Standard Algebraic Notation, the server validates it, runs the AI search, and returns the engine's reply — all within a couple of seconds at the search depths I am using.

This project is deliberately classical in its design. I am not using any machine learning here. The goal was to understand the foundations — the search algorithms, the evaluation heuristics, the move ordering tricks — before moving on to neural approaches. I think that context matters a lot when you later try to understand why AlphaZero works the way it does.

1.1 Game Theory in Chess

Chess is a deterministic, perfect-information, zero-sum game. This means both players have complete knowledge of the game state, there's no randomness, and one player's gain is exactly the other's loss. Such games are well-suited to minimax search.

2. System Architecture

The application follows a straightforward Client-Server architecture. The server handles all game logic and AI computation; the client handles rendering and user input. Keeping these concerns separated made the code much easier to reason about and debug.

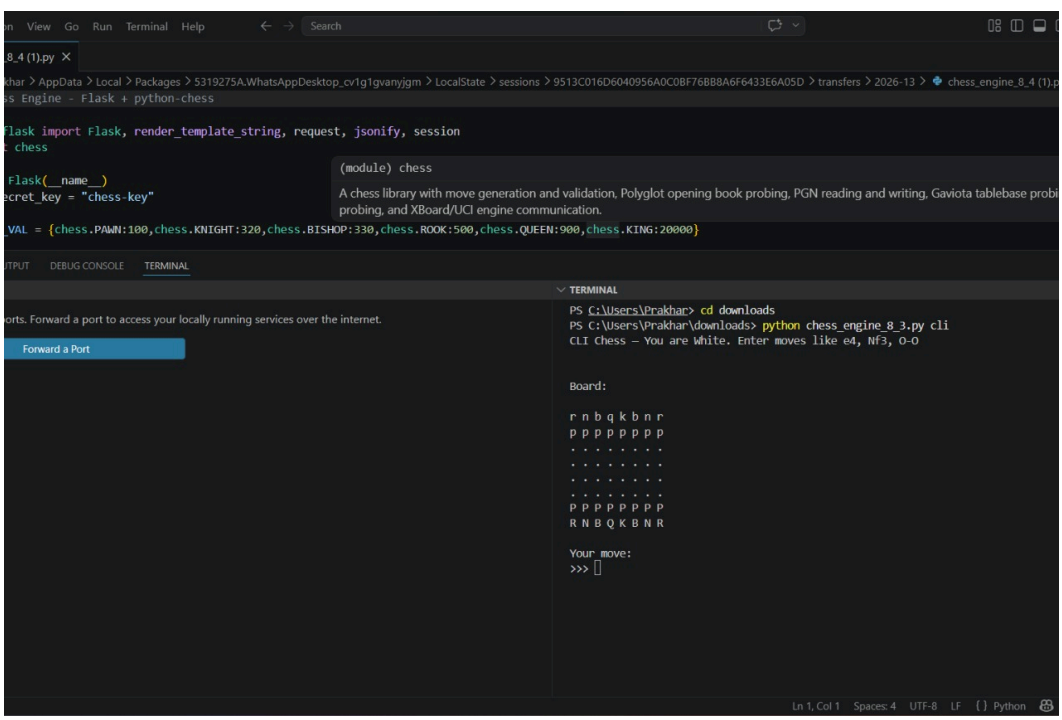
2.1 The Backend (Flask + Python)

Flask felt like the right choice here — it is minimal, stays out of the way, and lets me focus on the chess logic rather than framework boilerplate. The server exposes three endpoints. The root route (/) serves the entire frontend as a single rendered HTML string. The /move endpoint is the main workhorse: it receives the player's move as JSON, validates it with python-chess, calls the AI search, and returns the updated board state along with the engine's chosen move. The /reset endpoint clears the session and restarts the game.

Game state is stored in Flask's server-side session between requests. Specifically, I am storing the board's FEN (Forsyth-Edwards Notation) string, which is a compact encoding of the full board position, and the move history as a list of SAN strings. Reconstructing a chess.Board object from a FEN string on each request is essentially free, so there is no meaningful overhead from this approach.

"Command Line Interface (CLI) Integration"

The chess engine was enhanced with a Command Line Interface (CLI) in addition to the existing Flask-based web application. This allows users to play directly in the terminal without requiring a browser. The CLI mode uses the same underlying engine logic, including minimax and alpha-beta pruning, ensuring consistent gameplay across both interfaces. Users can input moves using standard algebraic notation, and the engine responds in real time. This dual-interface design improves accessibility and flexibility of the application.



2.2 The Frontend (Vanilla JavaScript + Chess.js)

I deliberately kept the frontend as simple as possible. It is a single HTML file with embedded CSS and JavaScript — no React, no build step, no dependencies beyond Chess.js loaded from a CDN. Chess.js handles client-side move validation so the user gets immediate feedback if they type an illegal move, without waiting for a round trip to the server.

The board is rendered as an 8x8 CSS grid. Each square is a div element, and pieces are displayed using Unicode chess symbols (♙, ♟, ♖, etc.) styled with text-shadow to differentiate the white and black pieces visually. When the engine responds, the last-moved squares are highlighted by toggling CSS classes. The move history panel updates in PGN-style numbering after every half-move. Nothing fancy, but it works cleanly and loads instantly.

3. AI Search Strategy

Before getting into the implementation details, it is worth establishing the theoretical context, because these algorithms are not arbitrary — they follow directly from the mathematical properties of the game itself.

3.1 Chess as a Zero-Sum Game

Chess is a zero-sum, perfect-information, two-player game. "Zero-sum" means that every point of advantage one player gains is an exactly equal loss for the other. There is no middle ground where both sides benefit. "Perfect information" means both players can see the entire board state at all times — no hidden cards, no fog of war.

These properties have a crucial implication: we can represent the quality of any board position with a single number. Positive scores favor White; negative scores favor Black. White's rational goal is to maximize this number; Black's is to minimize it. This adversarial framing maps perfectly onto the Minimax algorithm, which is essentially just a formalization of that logic.

3.2 Why Brute Force is Impossible — The Shannon Number

The naive approach would be to just search every possible game from the current position and pick the path that leads to a win. The problem, of course, is that this is computationally impossible. The Shannon Number — the estimated number of possible chess games — is approximately 10^120. To put that in perspective, it is many orders of magnitude larger than the number of atoms in the observable universe. A computer that evaluated one board position per nanosecond and ran for the entire age of the universe would not even scratch the surface of that search space.

This is why every practical chess engine, including mine, relies on intelligent search strategies rather than exhaustive enumeration. The question is not "how do I search everything?" — it is "how do I find a good move without searching very much?" Alpha-Beta Pruning is the classical answer to that question.

3.3 Framing It as a State Space Search

More formally, the AI's task is a state space search problem. A "state" is a board position encoded as a FEN string. An "action" is a legal move that transitions from one state to another. The engine's job is to search through the reachable states from the current position — the state space — and identify the action that leads to the best outcome within a given search depth. Minimax is the search algorithm; the evaluation function is how we judge the quality of a state; and Alpha-Beta Pruning is the mechanism that keeps the search tractable.

3.4 The Minimax Algorithm

Minimax works by building a game tree. Starting from the current position, it generates all legal moves, applies each one to produce a new board state, and recurses. At each level, the algorithm alternates roles: when it is White's turn, it picks the move with the highest score (the Maximizer); when it is Black's turn, it picks the move with the lowest score (the Minimizer). This alternation continues until the search hits its depth limit or reaches a terminal game state.

At the leaves of the tree — when depth reaches zero, or the game is over — the static evaluation function is called to assign a numeric score to the position. That score then bubbles back up through the tree. Each Maximizer node propagates the highest score it saw from its children; each Minimizer node propagates the lowest. By the time we get back to the root, we know which move leads to the best guaranteed outcome against a perfectly rational opponent.

In the code, the signature is *minimax(board, depth, alpha, beta, maximizing)*. The *depth* parameter decrements on each recursive call. When it hits zero, we hand off to Quiescence Search rather than calling the evaluation function directly — more on that below.

3.5 Alpha-Beta Pruning

Alpha-Beta Pruning is the single most impactful optimization in classical chess AI. It runs on top of Minimax without changing the result — the move it ultimately selects is identical to what plain Minimax would choose — but it avoids evaluating huge portions of the search tree that cannot possibly affect the outcome.

The idea is straightforward: as we search, we track two values. Alpha is the best score the Maximizer has found so far on the current path; Beta is the best score the Minimizer has found so far. Whenever Beta falls at or below Alpha, we know the current subtree is irrelevant — the Minimizer would never allow the game to reach this branch, because the Maximizer already has a better option elsewhere. We immediately stop exploring and move on. This is called a "cutoff."

In the best case — when the best moves are always examined first — Alpha-Beta reduces the effective branching factor from b to roughly the square root of b. For chess, where the average branching factor is around 35, this means the engine can effectively search twice as deep in the same amount of time compared to plain Minimax. That is the difference between a depth-2 and depth-4 search — the latter is dramatically stronger.

3.6 Quiescence Search — Avoiding the Horizon Effect

Here is a subtle but important problem with any fixed-depth search: what if the last move before you hit your depth limit was a Queen capture? The board looks great for whoever made that capture — but you are not seeing that the Queen can be immediately recaptured on the very next

move. This is called the "Horizon Effect," and it causes engines to make all kinds of bizarre-looking blunders.

Quiescence Search fixes this by refusing to evaluate a position at depth zero if it is tactically unstable. Instead of immediately calling the evaluation function when depth reaches zero, it extends the search to resolve all captures — and in some implementations, checks — until we reach a "quiet" position where there are no more forced tactical exchanges. Only then does it evaluate.

The implementation in my engine uses the same alpha-beta framework as the main search. There is a "stand-pat" check at the start: if the current static evaluation already beats beta, we return immediately without examining any captures, since the position is already too good for the current player and the opponent would not have allowed it. This is a standard fail-soft cutoff and it prevents the quiescence search from blowing up in highly tactical positions.

4. Heuristic Evaluation Model

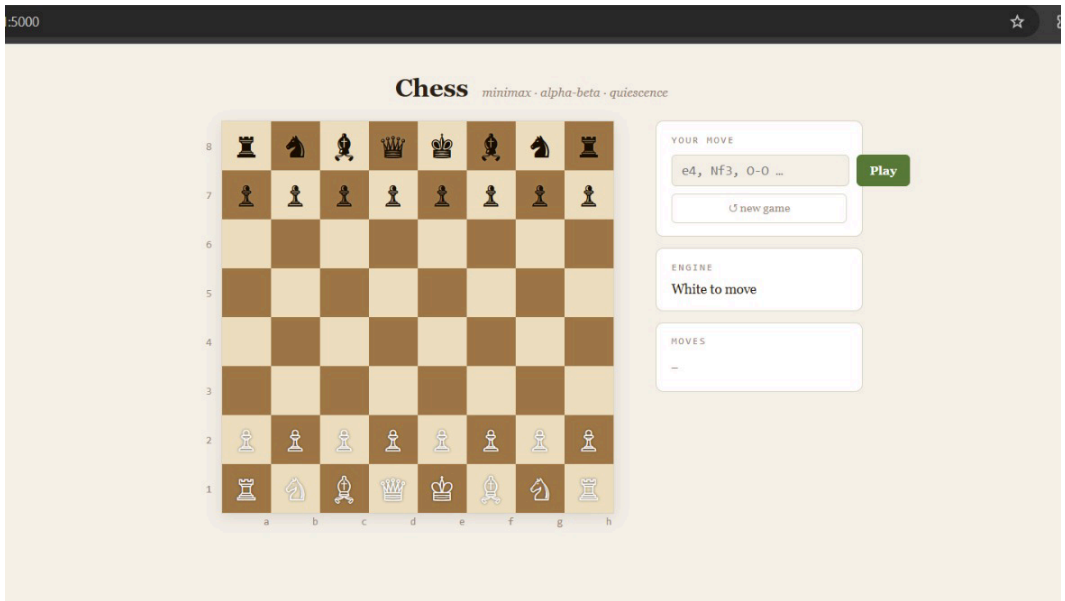
The *evaluation function is where the "intelligence" of the engine actually lives.* Search tells you which future positions are reachable; evaluation tells you which ones are good. A weak evaluation function will produce a weak engine regardless of how deep the search goes. Getting this right matters.

4.1 Material Counting

The foundation is simple: count each side's pieces and assign point values. These values are expressed in centipawns — hundredths of a pawn — which is the standard unit in chess programming. The values I am using are fairly conventional:

- **Pawn —** 100 centipawns (the baseline unit)
- **Knight —** 320 centipawns
- **Bishop —** 330 centipawns (slightly preferred over a Knight in most positions)
- **Rook —** 500 centipawns
- **Queen —** 900 centipawns
- **King —** 20,000 centipawns (high enough that the engine never voluntarily loses it in non-terminal evaluation)

The net material score is the sum of White's piece values minus the sum of Black's piece values. A positive result favors White; negative favors Black. This alone goes a long way — a two-pawn advantage is almost always winning at the amateur level — but it completely ignores where the pieces are, which turns out to matter a great deal.



Img for 4.1

4.2 Piece-Square Tables

A Knight on a1 — stuck in the corner, controlling only two squares — is strategically very different from a Knight on d4, sitting in the center of the board and controlling eight squares. Yet plain material counting assigns them identical value. Piece-Square Tables (PSTs) fix this by adding a positional bonus or penalty to each piece based on which square it occupies.

Each piece type has its own 8x8 table of 64 values, hand-tuned to reflect established chess principles. The Knight PST heavily penalizes corner and edge squares and rewards central squares. The Pawn PST encourages advancement toward promotion, penalizes doubled pawns slightly, and rewards central pawn control. The Queen PST tends to keep her closer to home early on, avoiding premature development into the center where she can be chased around by minor pieces. The Rook PST rewards open files and the seventh rank. The Bishop PST rewards the two long diagonals.

For White, I read the PST from the top-left corner of the table (rank 8, file a) down to rank 1. For Black, the table is mirrored vertically, so the same positional logic applies symmetrically. Each piece's total contribution to the score is its material value plus its PST bonus for its current square.

Img for 4.2

4.3 Endgame Detection and the King Activity Switch

One of the more interesting things I had to handle was the King's changing role across game phases. In the opening and middlegame, the King should stay safe — castled, tucked behind pawns, far from the action. If you look at the middlegame King PST, it assigns strong penalties to central squares and mild bonuses to the corners where a castled King typically sits. That is correct behavior for most of the game.

But in the endgame, all of that inverts. With fewer pieces on the board, mating threats largely disappear, and the King needs to become active. A King that hides on g1 while the opponent's King marches through the center to support a passer is playing the endgame completely wrong. The endgame King PST therefore awards strong bonuses to central squares and penalizes passivity.

The engine detects the endgame transition using the *is_endgame()* function, which checks two conditions: either all Queens have been traded off, or the total number of minor pieces (Knights, Bishops, Rooks) on both sides is two or fewer alongside the Queens. When either condition is met, the King's PST switches automatically from the middlegame table to the endgame table. This is a simplified form of what more sophisticated engines call "tapered evaluation," where the transition is gradual rather than binary, but it works well enough for this engine's depth of play.

5. Performance Optimizations

5.1 MVV-LVA Move Ordering

I mentioned earlier that Alpha-Beta Pruning works best when the best moves are examined first. In practice, this means the order in which we feed moves to the search matters enormously. If we get lucky and examine the best move first, the algorithm quickly raises the alpha bound and prunes most subsequent branches. If we examine moves in a bad order, we get far fewer cutoffs and the search slows down dramatically.

The heuristic I am using for move ordering is called MVV-LVA — Most Valuable Victim, Least Valuable Aggressor. For any capture move, the score is computed as:

```
Score = Value(Captured Piece) - Value(Capturing Piece)
```

So a Pawn capturing a Queen gets a very high score (900 − 100 = 800), and a Queen capturing a Pawn gets a low score (100 − 900 = −800). Captures with high MVV-LVA scores are examined first. The intuition is simple: winning material exchanges are very likely to be among the best moves in a position, so looking at them first means we raise the alpha bound quickly and prune more of the tree.

Non-capture moves that deliver check receive a secondary bonus and are examined before quiet non-checking moves. This is because checks are forcing and often represent the best moves in tactical positions.

In practice, good move ordering makes an enormous difference. It can reduce the number of nodes the engine needs to evaluate by an order of magnitude for a given depth — which directly translates to either much faster search at the same depth, or deeper search within the same time budget. It is probably the cheapest performance improvement available in terms of implementation complexity vs. payoff.

6. Where This Could Go Next — ML Integration

Everything in this engine is symbolic AI. The knowledge — piece values, PST scores, endgame detection thresholds — was put there by a human (me). The engine does not learn anything; it just applies fixed rules as efficiently as possible. That is fine for the purposes of this project, but it is worth being honest about the limitations.

The most obvious ceiling is the evaluation function. My PSTs represent static averages — a Knight on d4 gets the same bonus regardless of whether the position is closed (where Knights excel) or open (where Bishops tend to dominate). A neural network trained on millions of grandmaster games would learn these contextual patterns automatically, without anyone having to hand-code them.

6.1 Value Network

The evaluation function could be replaced with a Convolutional Neural Network (CNN) that takes the board position as input and outputs a scalar score. The standard encoding represents the board as a stack of binary planes — one plane per piece type per color, plus planes for castling rights, en passant, and move count. The CNN learns to recognize positional patterns across these planes that no hand-coded evaluation could capture: pawn structure dynamics, piece coordination, long-term strategic imbalances. This is essentially the Value Network from AlphaZero.

6.2 Policy Network

Separately, a Policy Network could replace move ordering entirely. Rather than using MVV-LVA to rank moves, the Policy Network outputs a probability distribution over all legal moves, trained to match the moves chosen by strong players in a large game database. The engine would then focus its Minimax search on the top-k moves suggested by the network, dramatically reducing the branching factor. Instead of searching 35 branches at each node, you might only need to examine 5 or 6 without losing much strength.

6.3 Self-Play and Monte Carlo Tree Search

AlphaZero takes this one step further by throwing out Minimax entirely and replacing it with Monte Carlo Tree Search (MCTS), guided jointly by the Policy and Value networks. The engine starts with no chess knowledge whatsoever — not even the piece values I am using — and learns purely through self-play via reinforcement learning. Within 24 hours of training, it surpassed every previous chess engine in history. Understanding the Minimax-based approach in this project is, I think, a necessary prerequisite for really grasping why MCTS + neural guidance works so well. The conceptual scaffolding is the same; only the implementation differs.

6.4 Transposition Tables (Nearer-Term)

A more immediate improvement that does not require any ML would be adding a Transposition Table. Many different move sequences lead to the same board position — for example, 1.e4 e5 2.Nf3 Nc6 and 1.Nf3 Nc6 2.e4 e5 are different orderings of the same moves that produce the same position. Currently my engine re-evaluates these transpositions from scratch every time it encounters them, which is wasteful. A hash map keyed by Zobrist hash of the board state, storing the previously computed evaluation and the depth at which it was computed, would allow the engine to skip entire subtrees it has already seen. This would give a meaningful speedup with essentially zero loss in accuracy.

7. Conclusion

Building this engine taught me more about AI search than any textbook chapter did. There is something clarifying about watching an algorithm you wrote make a genuinely surprising tactical move — or, equally revealing, watching it blunder because the evaluation function does not understand a particular type of position. Both outcomes tell you something concrete about what the code is actually doing.

The technical takeaway is that a relatively small set of well-chosen algorithms can produce a program that plays chess at a competent amateur level: Minimax defines the search, Alpha-Beta makes it fast enough to be practical, Quiescence Search keeps it from making tactically embarrassing mistakes, and the evaluation function gives it strategic direction. Each piece builds on the one before it in a fairly clean way.

The broader takeaway — for an AIML portfolio specifically — is that these classical techniques are not obsolete. They are the conceptual foundation that modern neural chess engines (Leela Chess Zero, AlphaZero) are implicitly building on, just expressed in learned rather than hand-coded form. The Value Network is an evaluation function. The Policy Network is move ordering. MCTS is a probabilistic variant of tree search with alpha-beta-like pruning. Understanding this engine deeply means understanding why those neural systems are designed the way they are, which I think is a more useful thing to know than treating them as black boxes.

The full source code for the engine, including the Flask backend and the frontend, is available as a single Python file. The project is self-contained and runs out of the box with Flask and python-chess installed.